Lập trình mạng · Chương 3

# Bài toán ứng dụng: truyền video theo nhóm multicast

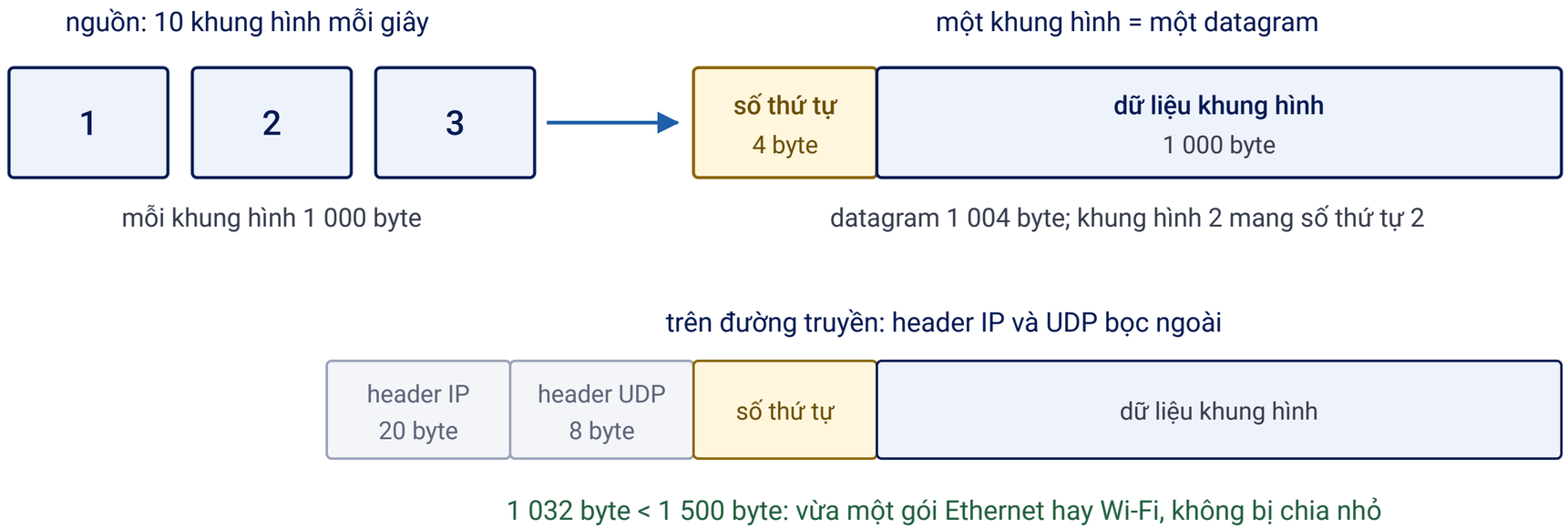
TS. Đặng Ngọc Hùng

# Video multicast: khung hình đến liên tục, có thể mất, máy nhận vào giữa chừng

1. Video là chuỗi **khung hình** liên tục, ví dụ 10 khung hình mỗi giây, tới nhiều máy nhận
2. UDP có thể **mất** hoặc **đảo thứ tự** datagram, và không báo
3. Máy nhận có thể **tham gia nhóm khi nguồn đang phát**

> **Bài toán:** đóng gói khung hình thế nào để máy nhận phát hiện được khung hình mất, và vẫn chạy tiếp khi mất?

# Mỗi khung hình thành một datagram, đứng đầu là số thứ tự 4 byte




**Đóng gói khung hình**: mỗi khung hình một datagram, gồm **số thứ tự** 4 byte và dữ liệu; datagram đủ nhỏ để đi trọn trong một gói.

# Máy nhận so số thứ tự: có khoảng trống thì bỏ qua và chạy tiếp




Máy nhận so **số thứ tự** với expected: lớn hơn là có khoảng trống, bỏ qua và chạy tiếp; khung hình đầu tiên nhận được là điểm bắt đầu.

# Nguồn ghi số thứ tự và khung hình vào một ByteBuffer, phát đều 10 khung hình mỗi giây

Java

```
for (int seq = 1; seq <= 100; seq++) {
    ByteBuffer buf = ByteBuffer.allocate(4 + FRAME_BYTES);
    buf.putInt(seq);              // số thứ tự 4 byte, big-endian
    buf.put(frame(seq));          // dữ liệu khung hình
    socket.send(new DatagramPacket(buf.array(), buf.position(), group, PORT));
    long next = start + seq * 100_000_000L;  // 10 khung hình mỗi giây
```

Kết quả chạy

```
sent frame 20 (1004 bytes)
sent frame 40 (1004 bytes)
sent frame 60 (1004 bytes)
sent frame 80 (1004 bytes)
sent frame 100 (1004 bytes)
```

# Máy nhận đếm khoảng trống theo số thứ tự, khung hình đến muộn bị bỏ

Java

```java
int seq = ByteBuffer.wrap(p.getData(), 0, 4).getInt();
if (expected == -1) {
} else if (seq > expected) {
    lost += seq - expected;
} else if (seq < expected) {
    System.out.printf("%s: late frame %d, dropped%n", name, seq);  // đến muộn: không còn giá trị
expected = seq + 1;
```

Kết quả chạy

```
r2: first frame 1
r2: gap 3 -> 6, lost 2 frame(s)
r2: gap 7 -> 9, lost 1 frame(s)
r2: gap 33 -> 35, lost 1 frame(s)
r2: gap 60 -> 62, lost 1 frame(s)
r2: gap 67 -> 69, lost 1 frame(s)
r2: gap 97 -> 99, lost 1 frame(s)
r2: received 93 frames, lost 7 (7.0%)
```

# Máy nhận vào khi đang phát bắt đầu từ khung hình đầu tiên nhận được

Java

```
socket.setSoTimeout(10_000);  // chờ nguồn tối đa 10 giây; sau khung hình đầu: 2 giây
if (expected == -1) {
    System.out.printf("%s: first frame %d%n", name, seq);  // tham gia khi đang phát: bắt đầu từ đây
    socket.setSoTimeout(2_000);
```

Kết quả chạy

```
r3: first frame 53
r3: received 48 frames, lost 0 (0.0%)
```

# Multicast không gửi lại khung hình mất: bỏ qua và chạy tiếp

1. Gửi lại cũng là gửi tới **cả nhóm**: mọi máy nhận khác nhận thêm dữ liệu thừa
2. Khung hình **đến muộn** hơn lúc cần hiển thị thì không còn giá trị
3. Muốn giảm ảnh hưởng của mất gói: nguồn gửi kèm **dữ liệu dự phòng**, không gửi lại

> **Nguyên tắc:** máy nhận video multicast bỏ khung hình mất và chạy tiếp.

# Câu hỏi 1

**Máy nhận vừa hiển thị khung hình 41; datagram tiếp theo nhận được mang số thứ tự 44. Đã mất bao nhiêu khung hình?**

A. 3 khung hình.

B. 2 khung hình: 42 và 43.

C. 1 khung hình.

D. Không xác định được, vì UDP không đánh số datagram.

# Câu hỏi 1

## Máy nhận vừa hiển thị khung hình 41; datagram tiếp theo nhận được mang số thứ tự 44. Đã mất bao nhiêu khung hình?

A. 3 khung hình.

B. 2 khung hình: 42 và 43.

C. 1 khung hình.

D. Không xác định được, vì UDP không đánh số datagram.

Sau khung hình 41, expected bằng 42. Khung hình 44 đến nên số khung hình mất là 44 − 42 = 2, gồm 42 và 43. Lấy 44 − 41 là đếm cả khung hình 44 vừa nhận. UDP không đánh số, nhưng giao thức của ứng dụng đã thêm số thứ tự.

# Câu hỏi 2

## Vì sao máy nhận video multicast không yêu cầu nguồn gửi lại khung hình mất?

A. Vì MulticastSocket không gửi được datagram unicast.

B. Vì UDP không cho máy nhận gửi datagram ngược về nguồn.

C. Vì số thứ tự 4 byte không đủ để đánh số khung hình gửi lại.

D. Vì khung hình gửi lại tới cả nhóm, làm các máy nhận khác nhận thêm dữ liệu thừa, và khung hình đến muộn không còn giá trị.

# Câu hỏi 2

## Vì sao máy nhận video multicast không yêu cầu nguồn gửi lại khung hình mất?

A. Vì MulticastSocket không gửi được datagram unicast.

B. Vì UDP không cho máy nhận gửi datagram ngược về nguồn.

C. Vì số thứ tự 4 byte không đủ để đánh số khung hình gửi lại.

**D. Vì khung hình gửi lại tới cả nhóm, làm các máy nhận khác nhận thêm dữ liệu thừa, và khung hình đến muộn không còn giá trị.**

Nguồn gửi tới nhóm, nên khung hình gửi lại tới mọi máy nhận, kể cả máy đã có; khung hình đến muộn hơn lúc hiển thị cũng vô ích. Máy nhận vẫn gửi được datagram unicast về nguồn, nên B sai.

# Tổng kết

1. **Đóng gói khung hình**: mỗi khung hình một datagram, đầu là **số thứ tự** 4 byte; datagram đi trọn trong một gói
2. Máy nhận so số thứ tự với expected: phát hiện khoảng trống, **bỏ khung hình mất**; vào giữa chừng thì bắt đầu từ khung hình đầu tiên
3. **Không gửi lại** trong multicast: gửi lại tới cả nhóm, khung hình đến muộn không còn giá trị

**Từ khóa cần nhớ** số thứ tự · đóng gói khung hình · khung hình · nhóm multicast

# Mở rộng

### Chủ đề nâng cao

- RTP và RTCP cho truyền media thời gian thực
- Forward error correction

### Bài tập mở rộng

Thêm thống kê tỉ lệ mất gói mỗi 5 giây ở máy nhận và cảnh báo khi vượt 10%.

Tiếp theo

# WebSocket: nâng cấp từ HTTP và giao tiếp hai chiều